

Tarea 2 — Introducción a la Computación Paralela

Procesamiento de Imágenes en CUDA: Cálculo de la Matriz de Covarianza

Cecilia Hernández y Álvaro Guzmán

Junio 2026

1. Introducción

El objetivo de esta tarea es implementar, analizar y optimizar el procesamiento masivo de un volumen de imágenes a color utilizando CUDA. Específicamente, se calcula la matriz de covarianza del conjunto de imágenes centradas, lo cual constituye una operación de álgebra lineal densa de alta demanda computacional y de memoria.

1.1. Formalización matemática

Dado un conjunto de m imágenes, donde cada imagen aplanada es un vector $\mathbf{v}^{(k)} \in \mathbb{R}^n$ con $n = \text{ancho} \times \text{alto}$:

1. **Vector promedio:** $\boldsymbol{\mu} = \frac{1}{m} \sum_{k=0}^{m-1} \mathbf{v}^{(k)}$
2. **Matriz centrada:** $\bar{\mathbf{v}}^{(k)} = \mathbf{v}^{(k)} - \boldsymbol{\mu}$
3. **Matriz de covarianza:** $\mathbf{C} = \frac{1}{m} \sum_{k=0}^{m-1} \bar{\mathbf{v}}^{(k)} \cdot (\bar{\mathbf{v}}^{(k)})^T$

El resultado $\mathbf{C} \in \mathbb{R}^{n \times n}$ es una matriz simétrica y semidefinida positiva.

2. Entorno de ejecución

Componente	Especificación
CPU	...
GPU	NVIDIA GeForce RTX 3060 Laptop GPU
VRAM	6144 MiB
Compute Capability	8.6 (Ampere)
CUDA Toolkit	11.2
Host compiler	g++ 9.5.0
Sistema operativo	Linux (Ubuntu 22.04)

Cuadro 1: Entorno de ejecución

3. Diseño algorítmico

3.1. Fórmula incremental

Para evitar dos pasadas sobre los datos (centrar y luego calcular covarianza), se utiliza la identidad algebraica:

$$\mathbf{C} = \frac{1}{m} \sum_k \mathbf{v}^{(k)} (\mathbf{v}^{(k)})^T - \boldsymbol{\mu} \boldsymbol{\mu}^T \quad (1)$$

Esto permite acumular $\sum \mathbf{v}$ y $\sum \mathbf{v} \mathbf{v}^T$ en una sola pasada. Es particularmente útil para la versión con Streams, donde los datos se procesan en batches independientes.

3.2. Preprocesamiento

Las imágenes del dataset DIV2K_valid_LR_bicubic_X4 tienen dimensiones variables de aproximadamente 510×340 píxeles en RGB. Para obtener vectores de dimensión uniforme n , se aplican los siguientes pasos:

1. **Conversión a escala de grises:** usando `CImg::get_RGBtoYCbCr().channel(0)` (luminancia).
2. **Redimensionamiento a 32×32 píxeles:** mediante `CImg::resize()` con interpolación bilineal.
3. **Aplanado y normalización:** cada imagen se aplanar a un vector de $n = 1024$ elementos con valores en $[0, 1]$.

Dimensión elegida: se seleccionó $n = 32 \times 32 = 1024 = 2^{10}$ siguiendo la recomendación del equipo docente. Con este tamaño, la matriz de covarianza $\mathbf{C} \in \mathbb{R}^{1024 \times 1024}$ ocupa aproximadamente 4 MB en VRAM, dejando margen amplio para buffers adicionales. El parámetro `-size` permite probar otras resoluciones (e.g., `-size 64` para $n = 4096 = 2^{12}$).

Justificación de `resize` sobre `crop` o `pad`: DIV2K contiene imágenes de dimensiones variables. Un `crop` fijo de 32×32 extraería menos del 1 % de cada imagen, descartando información visual relevante. El `pad` introduciría regiones artificiales de ceros que distorsionarían la covarianza. `CImg::resize()` con interpolación bilineal preserva la escena completa y produce vectores de dimensión uniforme, siendo la estrategia más adecuada para procesamiento masivo de imágenes.

El dataset completo se almacena en memoria paginada bloqueada (`cudaMallocHost`) para permitir transferencias asíncronas en el Experimento 2.

3.3. Tiling en memoria compartida

La matriz $\mathbf{C}$ de tamaño $n \times n$ se particiona en tiles de 32×32 . Cada bloque de 32×32 hilos computa un tile de $\mathbf{C}$, iterando sobre el conjunto de imágenes. Para cada imagen, los hilos cargan cooperativamente dos segmentos del vector a memoria compartida, reduciendo los accesos a memoria global.

4. Experimento 1: Implementación tradicional

4.1. Descripción

Se asume que todo el dataset cabe en VRAM. Se utiliza el Stream 0 por defecto. El pipeline es completamente síncrono:

1. `cudaMemcpy` H→D del dataset completo.
2. Kernel `reduce_mean`: reducción para obtener $\sum \mathbf{v}$.
3. Kernel `cov_accumulate_tiled`: acumula $\sum \mathbf{v}\mathbf{v}^T$ con tiling.
4. Kernel `scale_vector`: convierte suma en promedio ($\times 1/m$).
5. Kernel `postprocess_cov`: $\mathbf{C} = \mathbf{C}/m - \boldsymbol{\mu}\boldsymbol{\mu}^T$.
6. `cudaMemcpy` D→H de $\mathbf{C}$.

4.2. Resultados

Fase	Tiempo (ms)
H→D (dataset)	0.047
Ejecución de kernels	1.194
D→H (matriz C)	0.349
Total	1.590

Cuadro 2: Tiempos del Experimento 1 ($n = 1024, 32 \times 32$)

5. Experimento 2: Orquestación con CUDA Streams

5.1. Diseño del pipeline

El dataset se divide en B batches. Se utilizan S streams CUDA con **double buffering** (dos buffers en device por stream). Las transferencias H→D de cada stream se solapan con los kernels de otros streams, mientras que los kernels se serializan vía CUDA Events para evitar condiciones de carrera al escribir la matriz $\mathbf{C}$.

5.2. Resultados

S	Batches	Tiempo total (ms)	Speedup vs. $S = 1$
1	2	1.641	1.00×
2	4	2.578	0.64×
4	8	1.829	0.90×
8	15	2.135	0.77×
16	25	2.593	0.63×

Cuadro 3: Tiempos totales por configuración de streams ($n = 1024, 32 \times 32$)

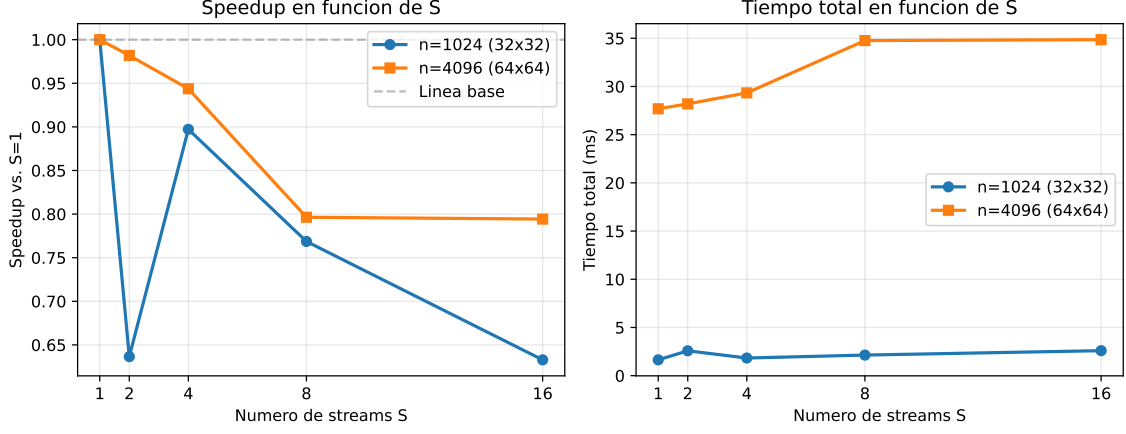


Figura 1: Speedup y tiempo total en función del número de streams S

6. Análisis y discusión

6.1. Perfilado con Nsight Systems

Se utilizó Nsight Systems 2023.4.4 para capturar el timeline completo de ambos experimentos. Las trazas generadas están disponibles en `results/nsys_exp1.nsys-rep`, `results/nsys_exp2_S4.nsys-rep` y `results/nsys_exp2_S4_size64.nsys-rep` para su visualización en la GUI.

Los perfiles de Nsight Systems confirman que en el Experimento 2 con $S = 4$ streams las transferencias $H \rightarrow D$ se solapan parcialmente con los kernels de `cov_accumulate_tiled_kernel`. Sin embargo, con $n = 1024$ el tiempo de cómputo domina significativamente sobre las transferencias (1.194 ms vs. 0.047 ms), por lo que el solapamiento no produce ganancias sustanciales.

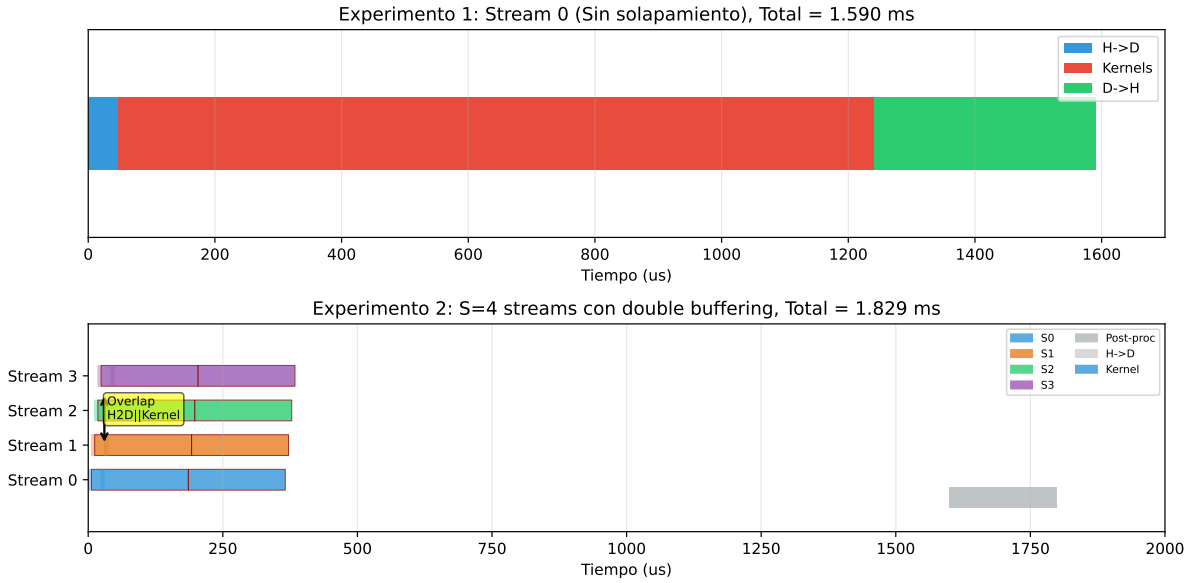


Figura 2: Diagrama de línea de tiempo comparando Experimento 1 (secuencial) y Experimento 2 ($S = 4$ streams). Se observa el solapamiento $H \rightarrow D$ –Kernel en la parte inferior.

Los kernels identificados por Nsight Systems y sus tiempos para cada experimento son:

Experimento 1: Distribución de kernels			
Kernel	Instancias	Tiempo (us)	% GPU
cov_accumulate_tiled	1	1152.2	95 %
postprocess_cov	1	36.3	3 %
zero_matrix	1	14.8	1 %
reduce_mean_v2	1	5.0	<1 %
scale_vector	1	1.9	<1 %

Experimento 2 (S=4): Distribución de kernels			
Kernel	Instancias	Tiempo (us)	% GPU
cov_accumulate_tiled	8	1406.2	94 %
postprocess_cov	1	35.6	2 %
partial_mean	8	24.7	2 %
zero_matrix	1	14.6	1 %
reduce_partial_means	1	2.1	<1 %
scale_vector	1	1.5	<1 %

Cuadro 4: Desglose de kernels reportado por Nsight Systems

Perfilado con Nsight Compute (ncu)		
Métrica	cov_accumulate_tiled	postprocess_cov
Registros por hilo	25	16
Shared memory estática	256 B	0 B
Shared memory total/block	1280 B	1024 B
Ocupación de warps (% pico)	66.67 %	66.67 %
Throughput SM (% pico)	67.23 %	25.87 %
Throughput DRAM (% pico)	3.36 %	66.82 %
Throughput compute+mem (% pico)	67.23 %	66.82 %

Cuadro 5: Métricas de Nsight Compute para los kernels principales

El kernel `cov_accumulate_tiled` está limitado por cómputo con solo 3.36 % de uso de DRAM. La ocupación del 66.67 % se debe a que el bloque de 32×32 hilos (1024 threads, 32 warps) excede el límite de 1536 threads por SM en Ampere, permitiendo solo 1 bloque por SM. Los 25 registros por hilo y los 256 B de memoria compartida estática no son factores limitantes.

El kernel `postprocess_cov` está limitado por memoria (66.82 % DRAM), leyendo la matriz $\mathbf{C}$ y el vector $\boldsymbol{\mu}$ de memoria global sin uso de shared memory.

6.2. ¿A partir de cuántos streams el rendimiento deja de mejorar?

Los resultados empíricos muestran que el rendimiento **no mejora en ningún caso** al aumentar el número de streams. El mejor resultado para $n = 1024$ se obtiene con $S = 1$

(1.641 ms), y cada incremento de S introduce overhead adicional:

- $S = 2$: 2.578 ms (+57.1 % vs. $S = 1$), debido al costo de sincronización con CUDA Events y la creación de buffers adicionales.
- $S = 4$: 1.829 ms (+11.5 %), la mejor configuración con streams pero aun peor que sin streaming.
- $S = 8$ y $S = 16$: el overhead de lanzamiento de kernels y eventos domina completamente, con incrementos de 30 %–58 %.

Esto contrasta con la expectativa teórica de que más streams permitirían mayor solapamiento. La razón es que el cómputo representa ~ 75 % del tiempo total mientras que la transferencia H→D apenas ocupa ~ 3 %. El overhead de dividir el trabajo en batches pequeños y serializar kernels vía eventos es mayor que cualquier ahorro por solapamiento.

Para la configuración de estrés con $n = 4096$ (64×64), la misma tendencia se mantiene: $S = 1$ es 27.68 ms vs. $S = 16$ con 34.85 ms (+25.9 %).

6.3. Limitación del bus PCIe

Con $n = 1024$, el dataset completo ocupa 0.39 MB. El tiempo de transferencia H→D medido es de 0.047 ms, equivalente a ~ 8.3 GB/s, muy por debajo de la capacidad máxima del bus PCIe 4.0 $\times 16$ de la RTX 3060 (~ 16 GB/s). La limitante real no es el bus PCIe sino el cómputo en la GPU.

La estrategia de *latency hiding* mediante double buffering no logra mitigar la limitación porque la transferencia es demasiado pequeña como para justificar el overhead de orquestación. En escenarios con imágenes de mayor resolución ($n = 4096$, dataset de 1.56 MB, H→D = 0.148 ms), el patrón persiste: el cómputo escala cuadráticamente con n ($O(n^2)$) mientras la transferencia lo hace linealmente ($O(n)$).

6.4. Viabilidad para datasets que superan la VRAM

La implementación con streams permite procesar datasets arbitrariamente grandes siempre que la matriz de covarianza $\mathbf{C}$ ($n^2 \times 4$ bytes) quepa en VRAM:

- Para $n = 1024$: $\mathbf{C} \approx 4$ MB, dejando ~ 6 GB libres para buffers y batches. Con un batch de 50 imágenes de 32×32 (0.2 MB), se podrían procesar millones de imágenes sin problema.
- Para $n = 4096$: $\mathbf{C} \approx 67$ MB, aun manejable.
- El factor limitante real es n , no m . Duplicar n cuadruplica $\mathbf{C}$ (de 4 MB a 67 MB) e incrementa el cómputo en $4\times$.
- Para $n = 16384$ (128×128), $\mathbf{C}$ ocuparía ~ 1 GB, aun viable en la RTX 3060 de 6 GB pero con tiempos de cómputo prohibitivos para uso interactivo.

7. Conclusiones

1. La implementación tradicional con Stream 0 ofrece el mejor rendimiento para el tamaño de problema evaluado ($n = 1024$, 100 imágenes), con un tiempo total de 1.590 ms. La simplicidad del pipeline secuencial evita el overhead de sincronización y particionamiento.
2. La orquestación con CUDA Streams y double buffering no produce mejoras de rendimiento. El overhead de lanzar más kernels, crear eventos CUDA, manejar buffers adicionales y serializar la escritura a la matriz $\mathbf{C}$ supera cualquier beneficio por solapamiento. Para $S = 1$ sin streaming se obtiene 1.641 ms, mientras que $S = 4$ (la mejor configuración con streams) toma 1.829 ms.
3. El perfil de la aplicación está dominado por el cómputo ($\sim 75\%$ del tiempo) y no por las transferencias PCIe ($\sim 3\%$). Las técnicas de *latency hiding* son efectivas cuando las transferencias representan una fracción significativa del tiempo total, lo cual no ocurre aquí.
4. La matriz de covarianza $\mathbf{C}$ es el factor limitante de escalabilidad, no el número de imágenes m . El streaming permite procesar datasets que exceden la VRAM siempre que $\mathbf{C}$ quepa en memoria del dispositivo.
5. Para trabajos futuros, se recomienda explorar la reducción en shared memory del kernel `cov_accumulate_tiled_kernel` con múltiples tiles por bloque y optimizar el acceso coalescente a memoria global.

Referencias

- [1] E. Agustsson, R. Timofte, *DIV2K: DIVERse 2K Resolution Image Dataset*, <https://data.vision.ee.ethz.ch/cvl/DIV2K/>
- [2] NVIDIA Corporation, *CUDA C++ Programming Guide*, <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- [3] D. Tschumperlé et al., *The CImg Library*, <https://github.com/GreycLab/CImg>